An Analysis of Feature engineering

Feature engineering

Feature engineering -- Part 1

Feature templates - implementing feature templates instead of coding new features Feature combinations - combinations that cannot be represented by a linear system Feature explosion can be limited via techniques such as regularization, kernel methods, and feature selection.

Feature engineering -- Part 2

Feature stores The feature store is where the features are stored and organized for the explicit purpose of being used to either train models (by data scientists) or make predictions (by applications that have a trained model). It is a central location where you can either create or update groups of features created from multiple different data sources, or create and update new datasets from those feature groups for training models or for use in applications that do not want to compute the features but just retrieve them when it needs them to make predictions. A feature store includes the ability to store code used to generate features, apply the code to raw data, and serve those features to models upon request. Useful capabilities include feature versioning and policies governing the circumstances under which features can be used. Feature stores can be standalone software tools or built into machine learning platforms.

Feature engineering -- Part 3

Multi-relational Decision Tree Learning (MRDTL) Multi-relational Decision Tree Learning (MRDTL) extends traditional decision tree methods to relational databases, handling complex data relationships across tables. It innovatively uses selection graphs as decision nodes, refined systematically until a specific termination criterion is reached. Most MRDTL studies base implementations on relational databases, which results in many redundant operations. These redundancies can be reduced by using techniques such as tuple id propagation.

Feature engineering -- Part 4

Clustering One of the applications of feature engineering has been clustering of feature-objects or sample-objects in a dataset. Especially, feature engineering based on matrix decomposition has been extensively used for data clustering under non-negativity constraints on the feature coefficients. These include Non-Negative Matrix Factorization (NMF), Non-Negative Matrix-Tri Factorization (NMTF), Non-Negative Tensor Decomposition/Factorization (NTF/NTD), etc. The non-negativity constraints on coefficients of the feature vectors mined by the above-stated algorithms yields a part-based representation, and different factor matrices exhibit natural clustering properties. Several extensions of the above-stated feature engineering methods have been reported in literature, including orthogonality-constrained factorization for hard clustering, and manifold learning to overcome inherent issues with these algorithms. Other classes of feature engineering algorithms include leveraging a common hidden structure across multiple inter-related datasets to obtain a consensus (common) clustering scheme. An example is Multi-view Classification based on Consensus Matrix Decomposition (MCMD), which mines a common clustering scheme across multiple datasets. MCMD is designed to output two types of class labels (scale-variant and scale-invariant clustering), and:

Feature engineering -- Part 5

Predictive modelling Feature engineering in machine learning and statistical modeling involves selecting, creating, transforming, and extracting data features. Key components include feature creation from existing data, transforming and imputing missing or invalid features, reducing data dimensionality through methods like Principal Components Analysis (PCA), Independent Component Analysis (ICA), and Linear Discriminant Analysis (LDA), and selecting the most relevant features for model training based on importance scores and correlation matrices. Features vary in significance. Even relatively insignificant features may contribute to a model. Feature selection can reduce the number of features to prevent a model from becoming too specific to the training data set (overfitting). Feature explosion occurs when the number of identified features is too large for effective model estimation or optimization. Common causes include:

Feature engineering -- Part 6

Feature engineering is a preprocessing step in supervised machine learning and statistical modeling which transforms raw data into a more effective set of inputs. Each input comprises several attributes, known as features. By providing models with relevant information, feature engineering significantly enhances their predictive accuracy and decision-making capability. Beyond machine learning, the principles of feature engineering are applied in various scientific fields, including physics. For example, physicists construct dimensionless numbers such as the Reynolds number in fluid dynamics, the Nusselt number in heat transfer, and the Archimedes number in sedimentation. They also develop first approximations of solutions, such as analytical solutions for the strength of materials in mechanics.

Feature engineering -- Part 7

Automation Automation of feature engineering is a research topic that dates back to the 1990s. Machine learning software that incorporates automated feature engineering has been commercially available since 2016. Related academic literature can be roughly separated into two types:

Feature engineering -- Part 8

featuretools is a Python library for transforming time series and relational data into feature matrices for machine learning. MCMD: An open-source feature engineering algorithm for joint clustering of multiple datasets. OneBM or One-Button Machine combines feature transformations and feature selection on relational data with feature selection techniques. OneBM helps data scientists reduce data exploration time allowing them to try and error many ideas in short time. On the other hand, it enables non-experts, who are not familiar with data science, to quickly extract value from their data with a little effort, time, and cost. getML community is an open source tool for automated feature engineering on time series and relational data. It is implemented in C/C++ with a Python interface. It has been shown to be at least 60 times faster than tsflex, tsfresh, tsfel, featuretools or kats. tsfresh is a Python library for feature extraction on time series data. It evaluates the quality of the features using hypothesis testing. tsflex is an open source Python library for extracting features from time series data. Despite being 100% written in Python, it has been shown to be faster and more memory efficient than tsfresh, seglearn or tsfel. seglearn is an extension for multivariate, sequential time series data to the scikit-learn Python library. tsfel is a Python package for feature extraction on time series data. kats is a Python toolkit for analyzing time series data.

Feature engineering -- Part 9

Alternatives Feature engineering can be a time-consuming and error-prone process, as it requires domain expertise and often involves trial and error. Deep learning algorithms may be used to process a large raw dataset without having to resort to feature engineering. However, deep learning algorithms still require careful preprocessing and cleaning of the input data. In addition, choosing the right architecture, hyperparameters, and optimization algorithm for a deep neural network can be a challenging and iterative process.